Les Tableaux Unidimensionnels et le Passage par Adresse en Langage C

Thème principal : Tableaux à une dimension, allocation mémoire contiguë, pointeurs et mécanisme d'Array Decay

Synthèse exécutive : Ce cours pose les fondements de la gestion de données séquentielles en langage C. Il explicite comment le compilateur réserve un espace continu en mémoire vive (stack) pour stocker un tableau unidimensionnel et comment le système accède directement à tout élément en temps constant O(1) grâce à une formule d'arithmétique système. Il décortique également le phénomène central d'Array Decay (la conversion automatique du nom d'un tableau en un pointeur vers son premier élément lors d'un passage en fonction), démontrant pourquoi il est techniquement impossible pour une fonction de deviner la taille d'un tableau reçu et pourquoi il est obligatoire de transmettre sa taille en paramètre séparé.

1. Représentation mémoire, contiguïté et calcul d'adresse O(1)

Lorsqu'un tableau est déclaré en C (par exemple `int tab[5];`), le compilateur alloue une plage continue d'adresses mémoire sur la pile (stack). Pour un tableau de 5 entiers de 4 octets, le bloc complet occupe exactement 20 octets consécutifs. L'accès à un élément via la syntaxe crochet `tab[i]` s'appuie sur la formule d'arithmétique système : Adresse(tab[i]) = Adresse_Base + (i * sizeof(type)). L'indice `i` représente le décalage (offset). Le premier élément se situe à un décalage de 0 octet par rapport au début, c'est pourquoi son indice est 0 (`tab[0]`). Tenter d'accéder à l'indice `N` (par exemple `tab[5]` pour une taille de 5) constitue une violation de mémoire (Buffer Overflow / Débordement de tableau) pouvant entraîner un comportement indéterminé ou un arrêt brutal du programme (Segmentation Fault).

- Tableau unidimensionnel : Structure de données statique et homogène stockant un nombre fixe d'éléments du même type.
- Contiguïté mémoire : Réservation d'un bloc ininterrompu de N * sizeof(type) octets en mémoire vive.
- Indexation de 0 à N-1 : L'indice i représente le décalage (offset) par rapport à l'adresse de base du tableau.
- Complexité constante O(1) : Accès direct à n'importe quelle case sans parcours préalable grâce à un calcul d'adresse mathématique.

Cas concret enseignant : *Le professeur a comparé le tableau à un meuble de rangement vertical comportant des tiroirs rigoureusement identiques et collés les uns aux autres, numérotés de 0 à N-1. Si l'on connaît l'emplacement du sol (l'adresse de base du meuble) et la hauteur exacte d'un tiroir, on calcule instantanément la hauteur exacte du tiroir i sans avoir besoin de mesurer ou d'ouvrir les tiroirs précédents un par un.*

2. Passage de tableaux aux fonctions et mécanisme d'Array Decay

Pour des raisons d'efficacité algorithmique, le langage C proscrit la copie intégrale de structures volumineuses lors d'un appel de fonction (pas de passage par valeur pour les tableaux). Lorsqu'un tableau est transmis à une fonction, son identifiant subit une 'dégradation' (Array Decay) : il se transforme en un simple pointeur `type*` contenant l'adresse mémoire du premier élément (`&tab;[0]`). Deux conséquences majeures en découlent : 1. Les modifications réalisées dans la fonction modifient directement le tableau original de la fonction appelante. 2. Dans le scope de la fonction appelée, l'opérateur `sizeof(tab)` renvoie la taille du pointeur (4 ou 8 octets selon l'architecture) et non la taille du tableau. Il est donc impératif de passer la taille effective du tableau sous forme de paramètre entier séparé (`int taille`).

- Array Decay : Conversion implicite et automatique du nom d'un tableau en un pointeur vers son premier élément (&tab;[0]) dès qu'il est passé en argument.
- Passage par adresse : Transmission de l'emplacement mémoire original au lieu d'une copie, évitant toute duplication de mémoire.
- Arithmétique des pointeurs : Équivalence stricte entre l'écriture crochet `tab[i]` et l'écriture pointeur `*(tab + i)`.

• Perte de taille : Une fonction recevant un tableau ne possède qu'un pointeur et ne peut plus déterminer dynamiquement le nombre d'éléments alloués.

Cas concret enseignant : *L'enseignant a expliqué que transmettre un tableau à une fonction ne revient pas à fabriquer une réplique exacte de sa maison dans le jardin de son ami (passage par valeur), mais plutôt à lui donner les coordonnées GPS précises de sa maison (passage par adresse). L'ami utilise ces coordonnées pour se rendre sur place et repeindre directement les murs du bâtiment d'origine.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou affiche des valeurs absurdes lorsque j'essaie de lire ou d'écrire dans tab[5] alors que mon tableau a été déclaré avec une taille de 5 (int tab[5];) ?
Réponse du professeur : Lorsqu'on déclare `int tab[5];`, les indices valides vont de `0` à `4`. L'indice `5` désigne la 6ème case, qui n'appartient pas au tableau. En C, il n'y a pas de contrôle automatique des bornes à l'exécution. Accéder à `tab[5]` provoque un débordement de mémoire (Buffer Overflow) : soit vous lisez une valeur 'poubelle' présente à cette adresse mémoire, soit le système d'exploitation interrompt brutalement le programme avec une Erreur de Segmentation (Segmentation Fault).

Question étudiant : Est-ce qu'on peut calculer la taille d'un tableau à l'intérieur d'une fonction avec l'expression sizeof(tab) / sizeof(tab[0]) ?
Réponse du professeur : Non, c'est un piège classique en partiel ! L'expression `sizeof(tab) / sizeof(tab[0])` ne fonctionne que dans le bloc de code où le tableau a été déclaré statiquement. Une fois transmis à une fonction, le tableau a subi l'Array Decay et `tab` n'est plus qu'un pointeur (`int*`). `sizeof(tab)` donnera la taille du pointeur (8 octets sur un système 64 bits) et non la taille totale du tableau. C'est pour cela qu'il faut obligatoirement passer un paramètre `int taille` à la fonction.

■ Exercice & Démonstration d'Application

Énoncé : Écrire une fonction C nommée `remplir_et_calculer_moyenne` qui prend en paramètres un tableau d'entiers et sa taille effective. La fonction doit demander à l'utilisateur de saisir chaque note au clavier pour remplir le tableau, puis calculer et retourner la moyenne arithmétique exacte des notes sous la forme d'un nombre à virgule (`double`). La fonction doit être sécurisée contre les tailles invalides.

Résolution pas à pas :
1. Prototype de la fonction : `double remplir_et_calculer_moyenne(int tab[], int taille)`. 2. Sécurité : Vérifier si `taille <= 0`. Si c'est le cas, retourner `0.0` pour éviter toute division par zéro. 3. Initialisation de l'accumulateur : Déclarer une variable `double somme = 0.0;` pour accumuler les valeurs tout en conservant la précision décimale. 4. Boucle de saisie : Parcourir le tableau avec un indice `i` variant de `0` à `taille - 1`. Utiliser `scanf("%d", &tab;[i])` pour enregistrer chaque saisie utilisateur. 5. Calcul et retour : Diviser `somme` par `taille` et retourner le résultat `double`.

Résultat : #include double remplir_et_calculer_moyenne(int tab[], int taille) { if (taille <= 0) { return 0.0; } double somme = 0.0; for (int i = 0; i < taille; i++) { printf("Saisir l'élément [%d] : ", i); scanf("%d", &tab;[i]); somme += tab[i]; } return somme / taille; } // Exemple d'exécution (pour taille = 3, saisies : 12, 14, 16) : // Calcul : (12 + 14 + 16) / 3 = 42.0 / 3 = 14.000000

■ Auto-Évaluation Flash d'Examen

Q : Quel est l'indice du dernier élément accessible d'un tableau d'entiers déclaré par 'int t[10];' ?

R : L'indice du dernier élément est 9. Les indices d'un tableau de taille N vont strictement de 0 à N-1.

Q : Que contient exactement l'identifiant d'un tableau lorsqu'il est employé seul sans crochets dans un programme C ?

R : Il contient l'adresse mémoire du premier élément du tableau, soit la valeur équivalente à &tab;[0].

Q : Pourquoi la signature d'une fonction manipulant un tableau C doit-elle obligatoirement inclure sa taille en paramètre ?

R : En raison du mécanisme d'Array Decay, la fonction ne reçoit qu'un pointeur vers le premier élément et perd toute information sur la dimension d'origine du tableau. Il est donc impossible pour la fonction de connaître sa taille sans la recevoir explicitement.